

National University of Computing and Emerging Sciences



Information Security **Assignment 02**

Submitted By:
i21-2487 Kalsoom Tariq

Submitted To:
Dr. Sayed Qaiser Ali Shah

Submission Date: 10-November-2024

Table of contents

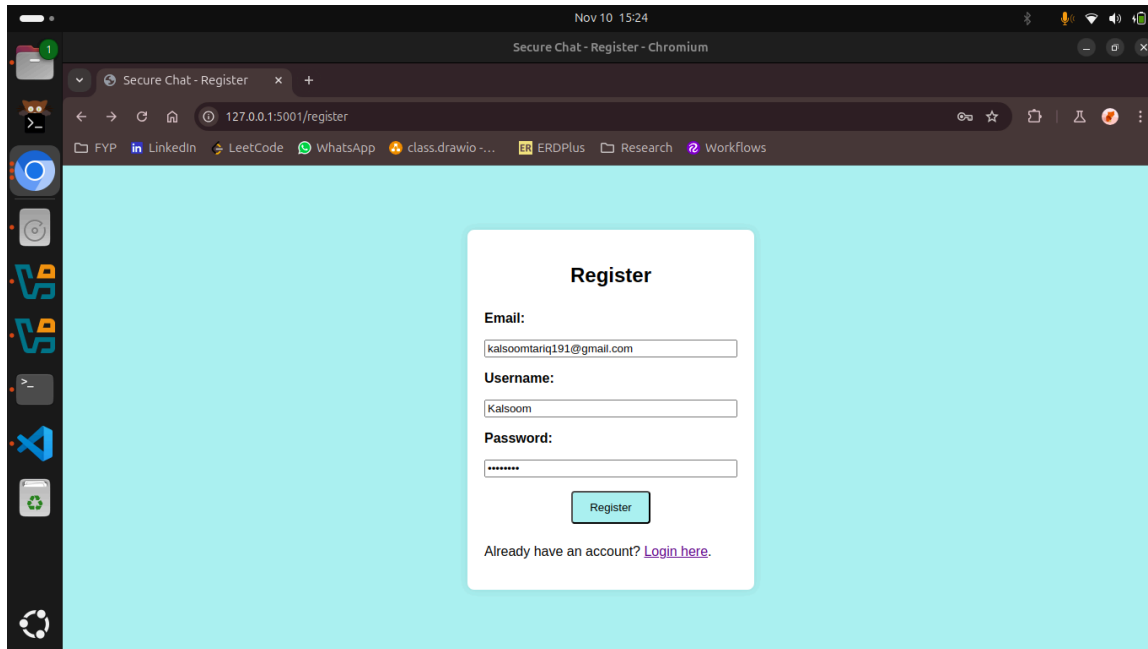
Functional Testing.....	2
Registration Testing.....	2
Login Testing.....	5
Security Testing.....	8
Hash Verification.....	8
Encryption Testing.....	12
Registration.....	12
Login.....	13
Chat.....	15

Functional Testing

Registration Testing

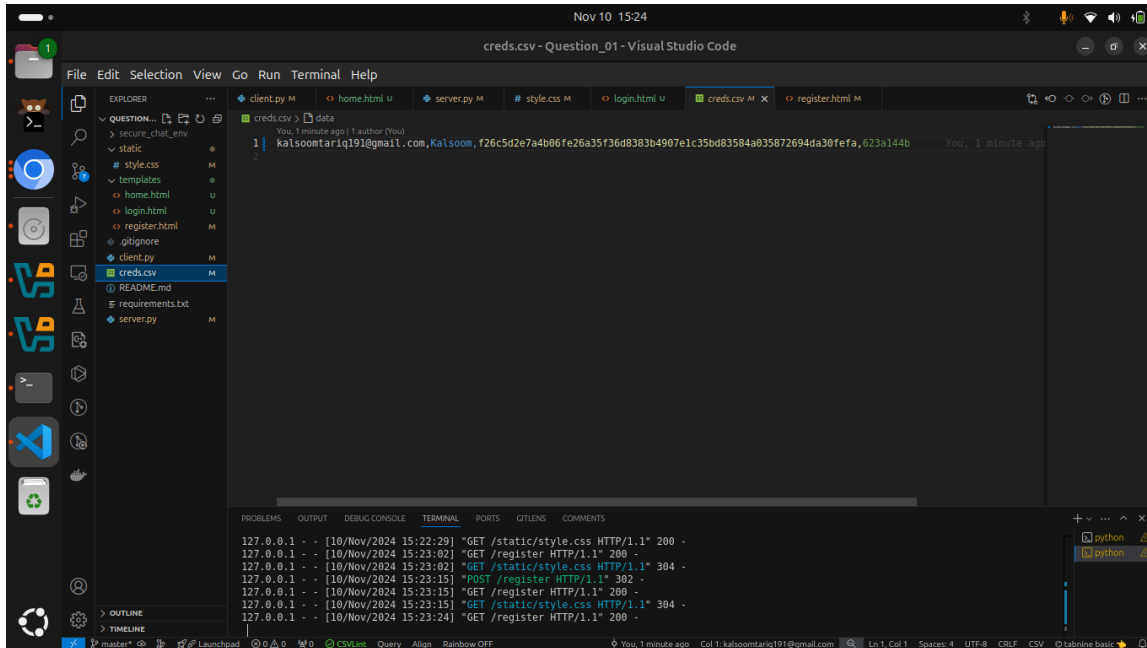
Verify that new users can register with a unique username and password (25 marks)

Step 1: Register a user with the name “Kalsoom” and any password.

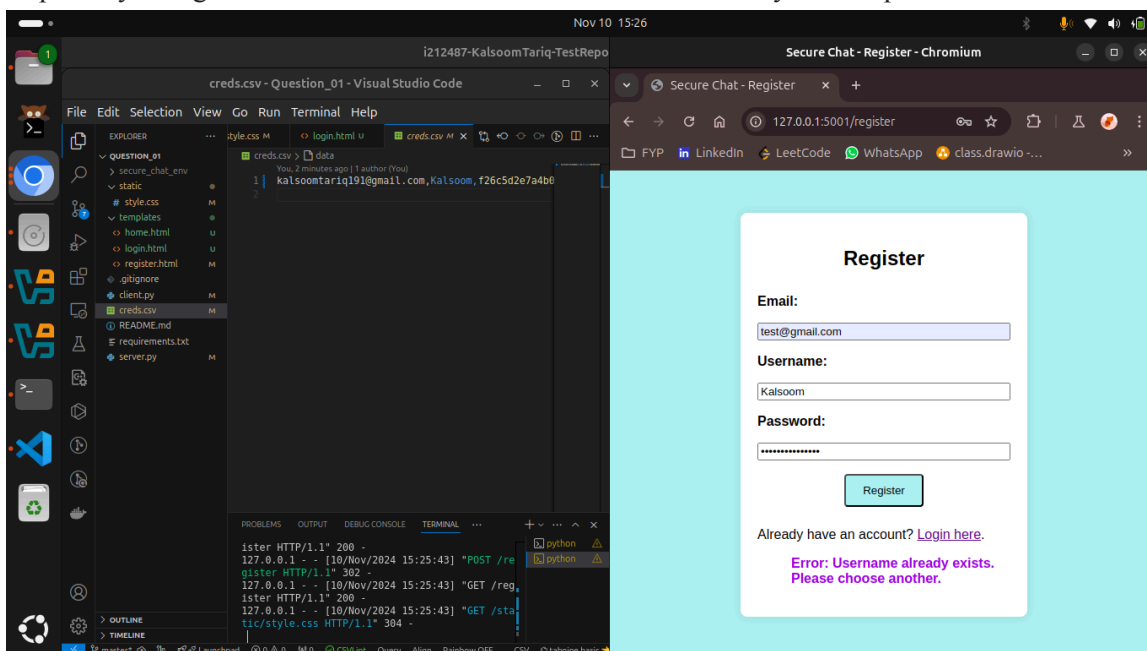


The screenshot shows a web browser window with the title 'Secure Chat - Register - Chromium'. The address bar displays '127.0.0.1:5001/register'. The page has a light blue background. In the center, there is a white registration form titled 'Register'. The form contains three input fields: 'Email:' with the value 'kalsoomtariq191@gmail.com', 'Username:' with the value 'Kalsoom', and 'Password:' with masked characters '*****'. Below these fields is a green 'Register' button. At the bottom of the form, there is a link that says 'Already have an account? [Login here.](#)'.

Verify the 'creds' file entry



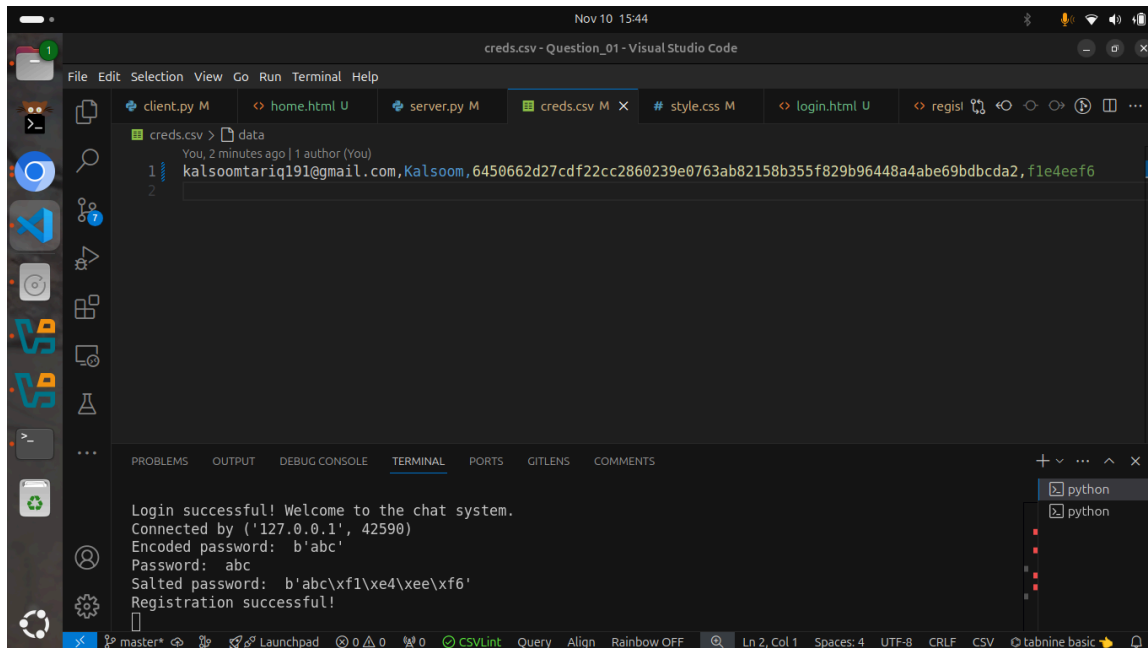
Step 2: Try to register with the same username 'Kalsoom' to verify the uniqueness.



We get an error from the server in this step as the username should be unique.

Ensure that the password is hashed correctly using SHA-256 and stored securely in the credential file along with salt value. (25 marks)

Open the credentials file and confirm the hash and salt value



Function to store salted password hash:

```

def store_credentials(email, username, password):
    # Check if username exists
    with open('creds.csv', mode='a+', newline='') as file:
        file.seek(0)
        csv_reader = csv.reader(file)
        is_unique = all(row[1] != username for row in csv_reader)

    if not is_unique:
        return False, "Error: Username already exists. Please choose
another."

    # Generate salt and hash password
    salt = os.urandom(4) # 32 bits salt
    salted_password = password.encode() + salt
    hashed_password = hashlib.sha256(salted_password).hexdigest()

    # Store credentials in creds.csv
    with open('creds.csv', mode='a', newline='') as file:
        csv_writer = csv.writer(file)
        csv_writer.writerow([email, username, hashed_password,

```

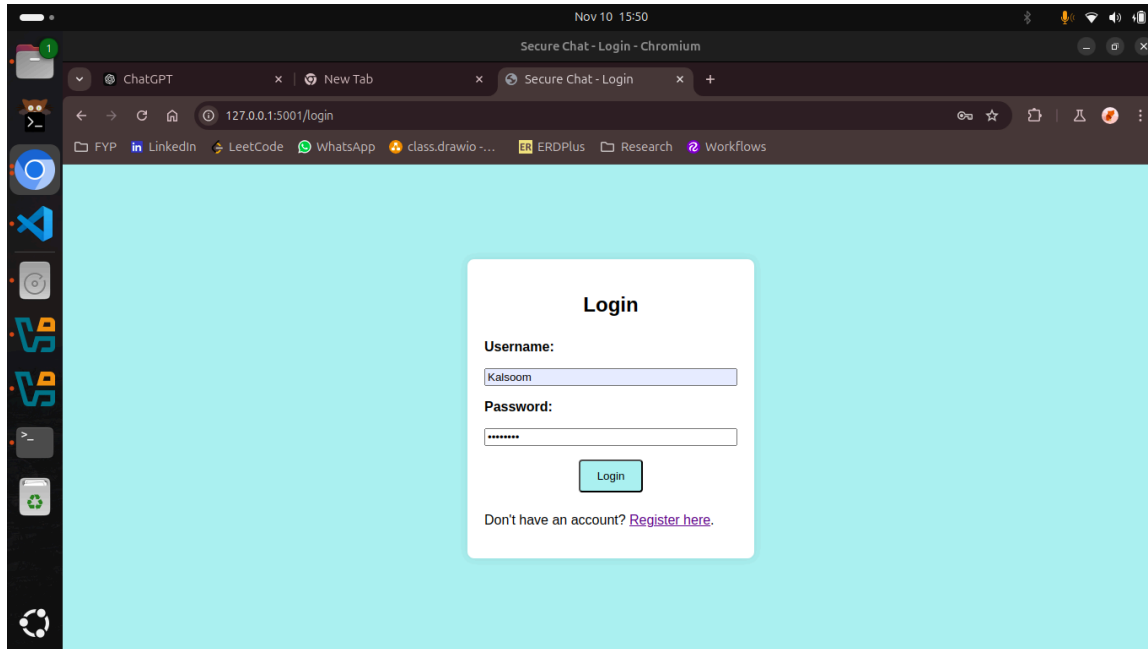
```
salt.hex()])
```

```
return True, "Registration successful!"
```

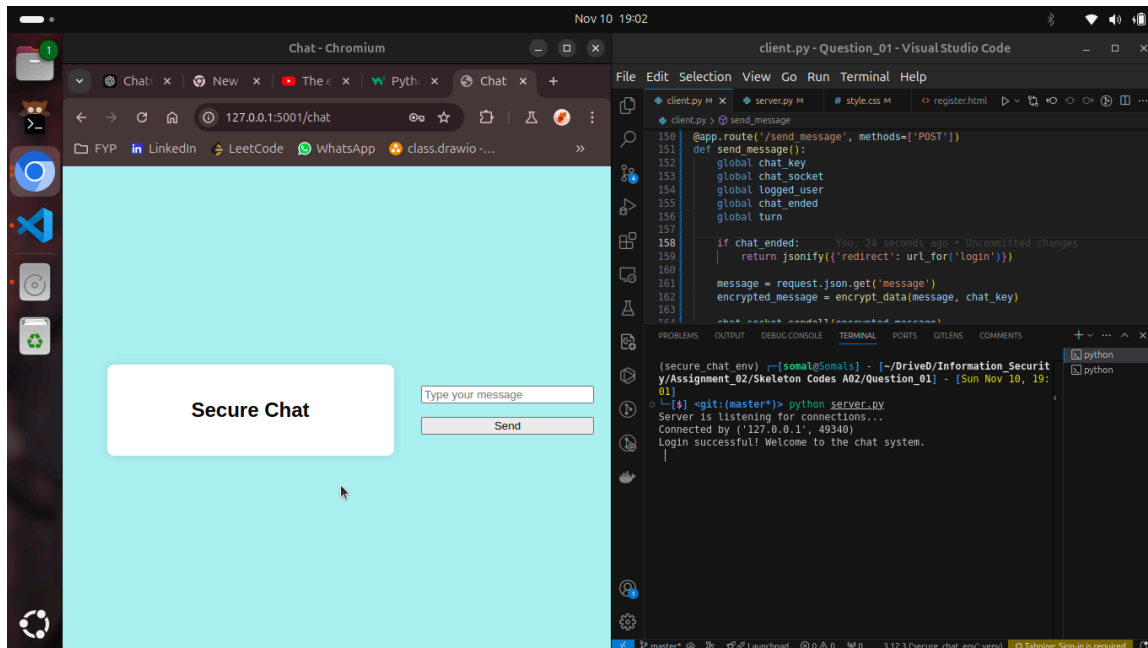
Login Testing

Test successful logins with valid credentials. (25 marks)

Enter the credentials used for registration i.e. username “Kalsoom” and password “12345678”

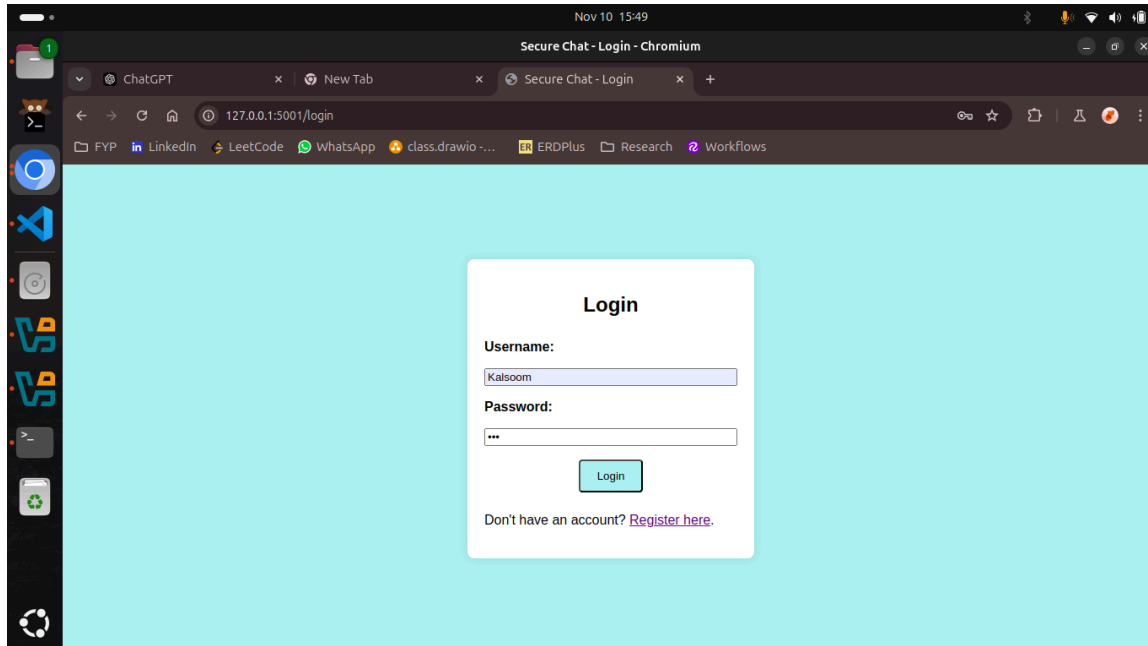


Enter login and you will be redirected to the secure chat application.

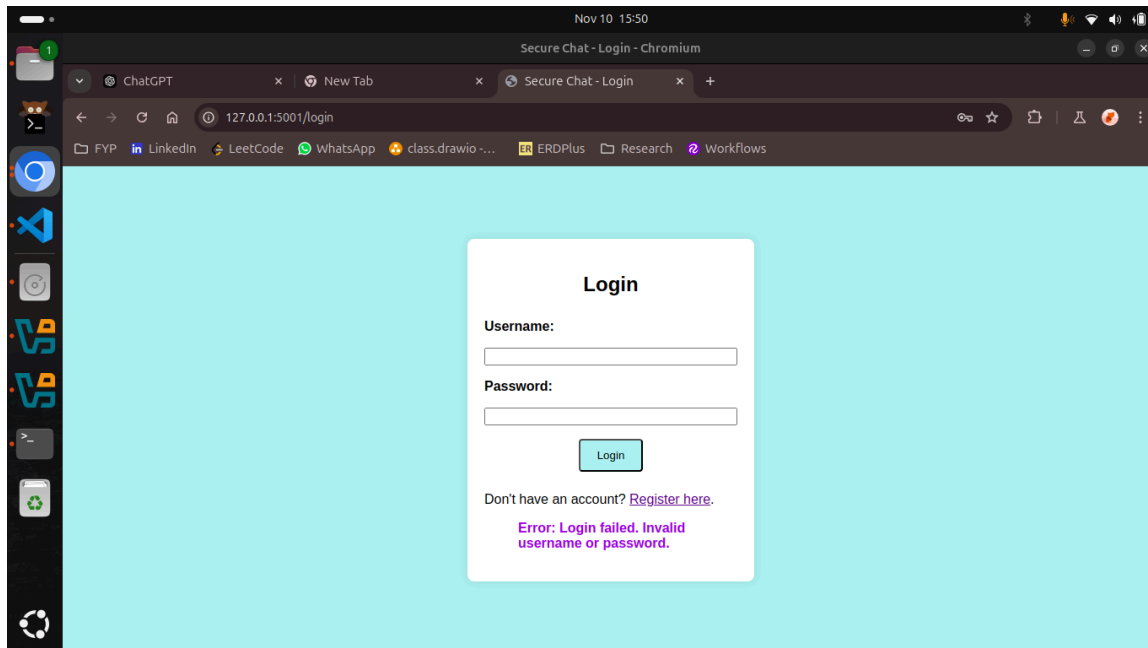


Test failed logins with incorrect credentials and ensure proper error messages are displayed. (25 marks)

Enter invalid credentials e.g. username “Kalsoom” and password “abc”



Click on Login and notice the error message.



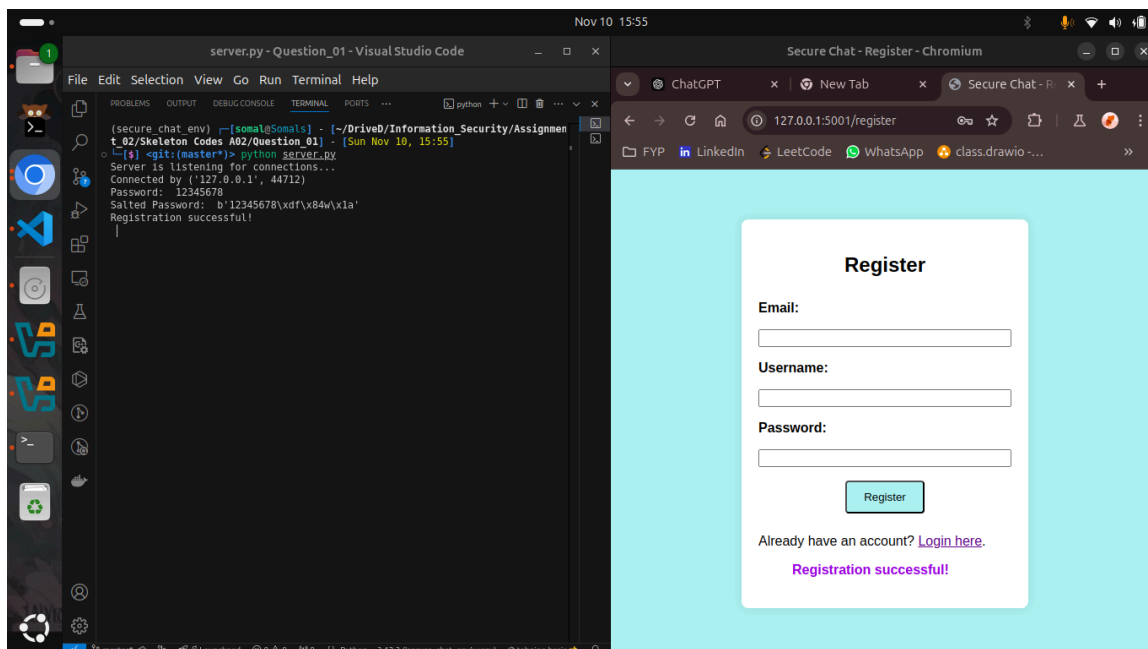
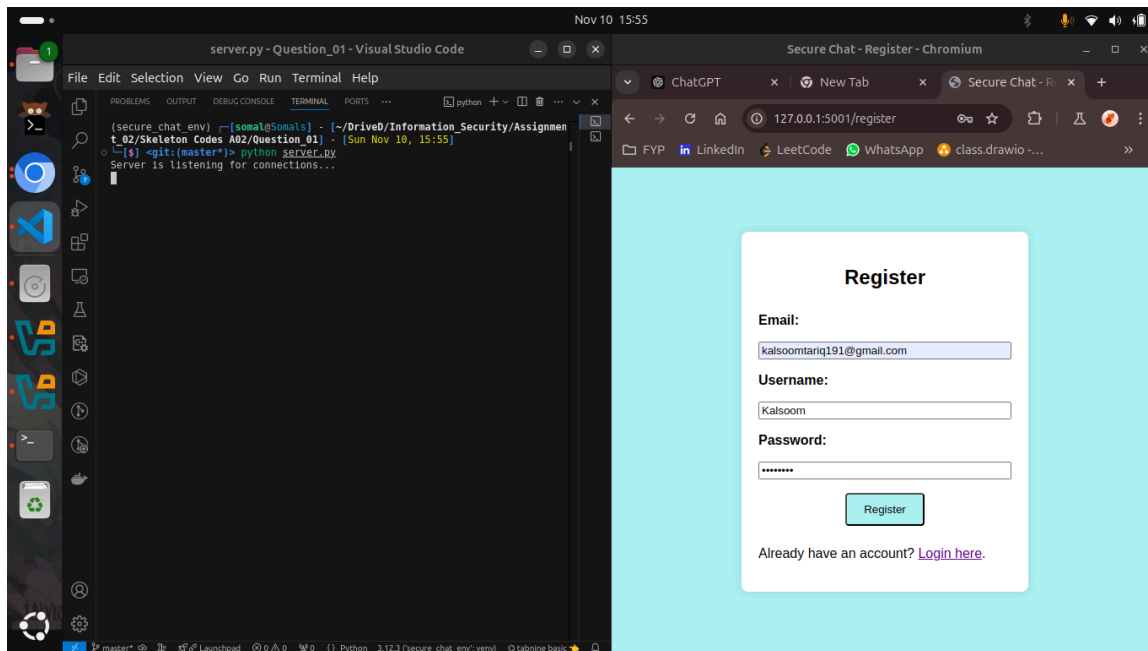
Security Testing

Hash Verification

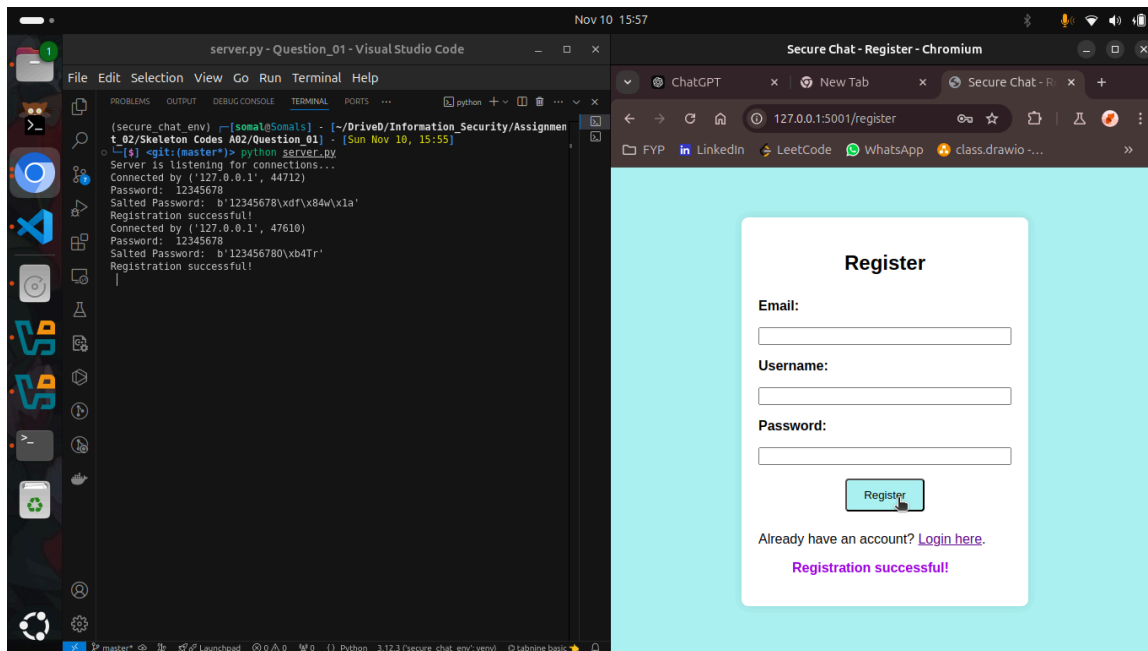
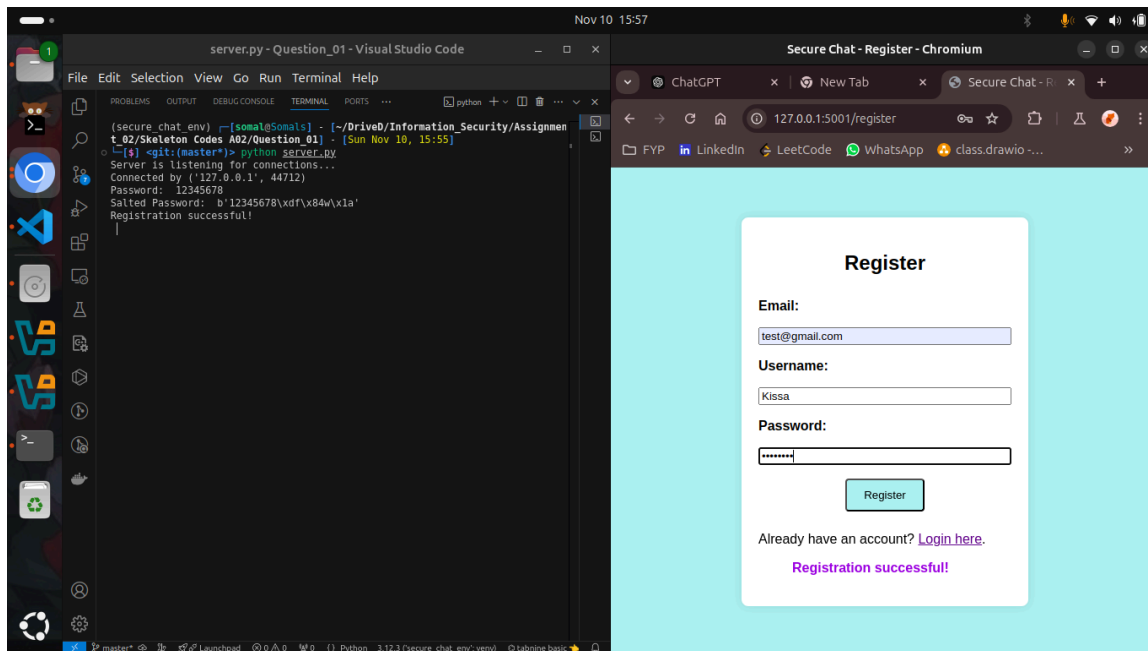
Verify that identical passwords generate different hashes due to random salting with SHA-256. (25 marks)

Let's use the same password "12345678" and observe two different hashes. I will log the password on the backend server to show that it's the same.

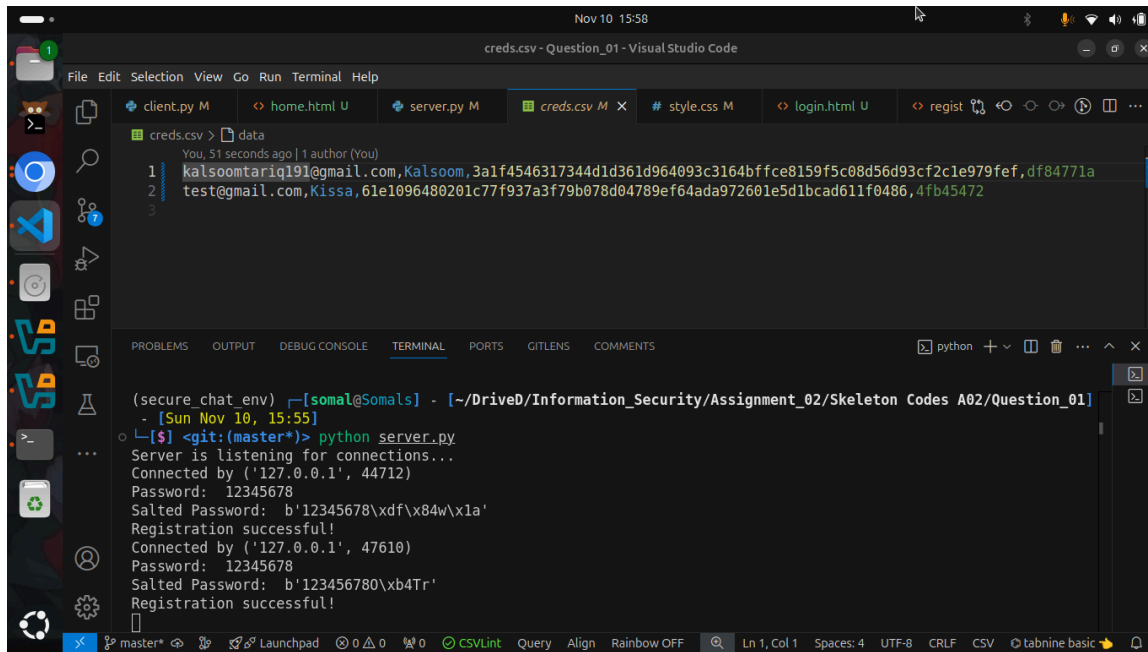
Using Account 1 with username "Kalsoom"



Using Account 2 with username “Kissa”



Let's see the 'creds' file



The screenshot shows the Visual Studio Code interface. The file explorer on the left shows a project named 'Question_01'. The file 'creds.csv' is open in the editor, showing two rows of data:

```
1 You, 51 seconds ago | 1 author (You)
2 kalsoomtariq191@gmail.com,Kalsoom,3alf4546317344d1d361d964093c3164bffe8159f5c08d56d93cf2c1e979fef,df84771a
3 test@gmail.com,Kissa,61e1096480201c77f937a3f79b078d04789ef64ada972601e5d1bcad611f0486,4fb45472
```

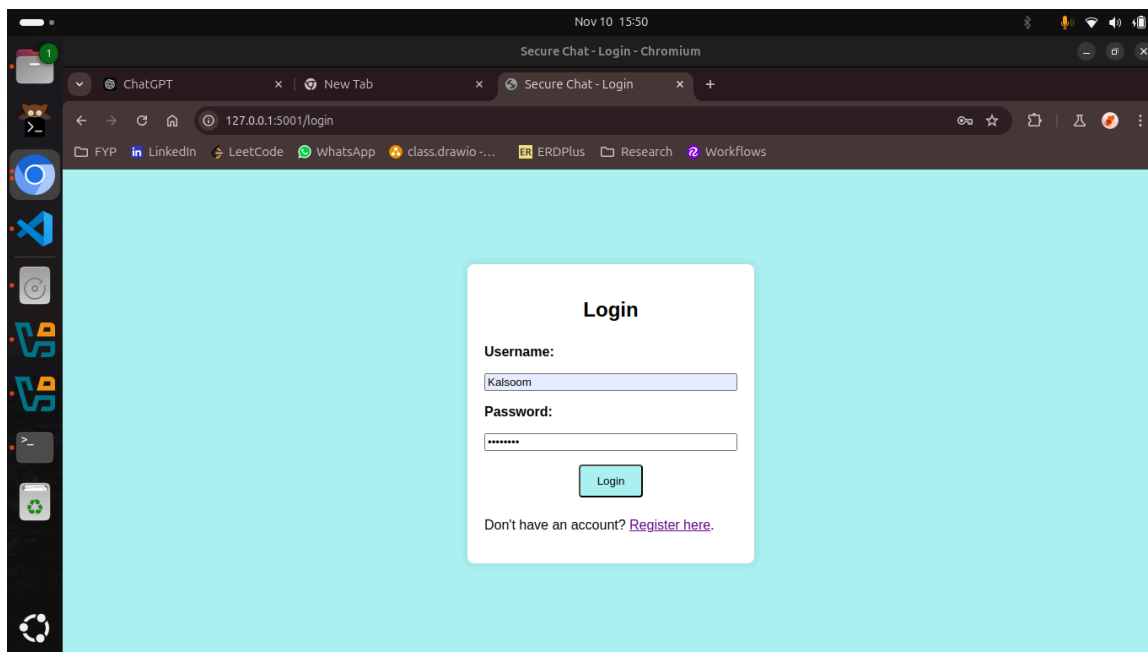
The terminal window at the bottom shows the output of running a Python server:

```
(secure_chat env) [somal@Somals] - [~/DriveD/Information_Security/Assignment_02/Skeleton Codes A02/Question_01]
- [Sun Nov 10, 15:55]
[!] <git:(master*)> python server.py
Server is listening for connections...
Connected by ('127.0.0.1', 44712)
Password: 12345678
Salted Password: b'12345678\xdf\x84w\x1a'
Registration successful!
Connected by ('127.0.0.1', 47610)
Password: 12345678
Salted Password: b'123456780\xb4Tr'
Registration successful!
```

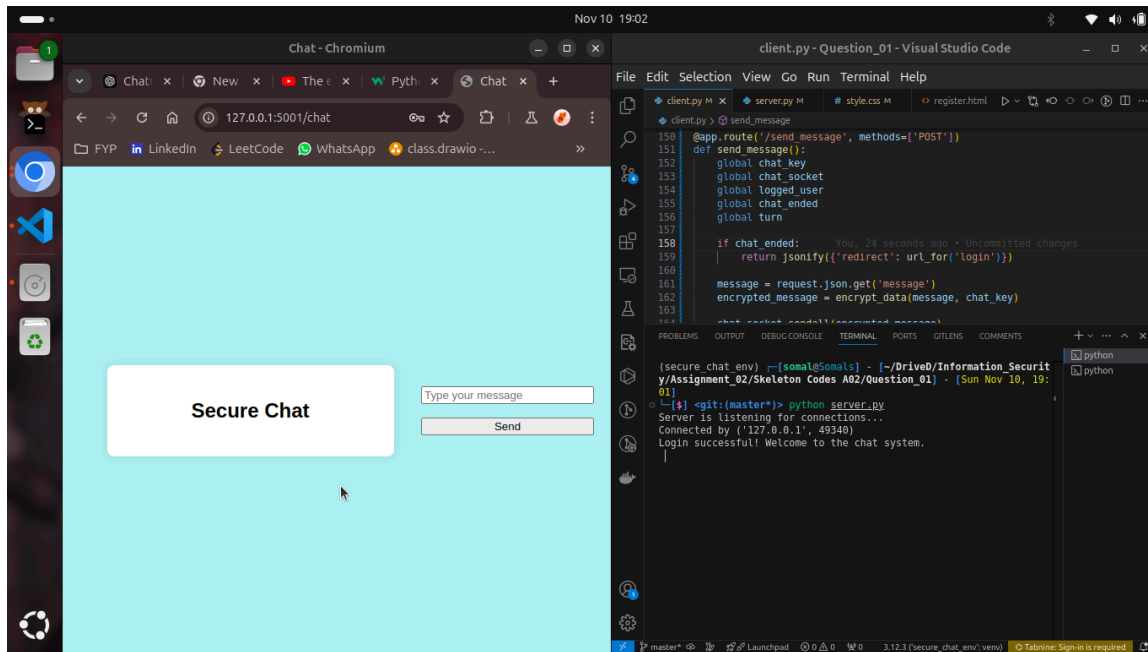
Due to different salt values, the generated hashes are different even though the password is the same.

Verify user passwords are verified during the login phase. (25 marks)

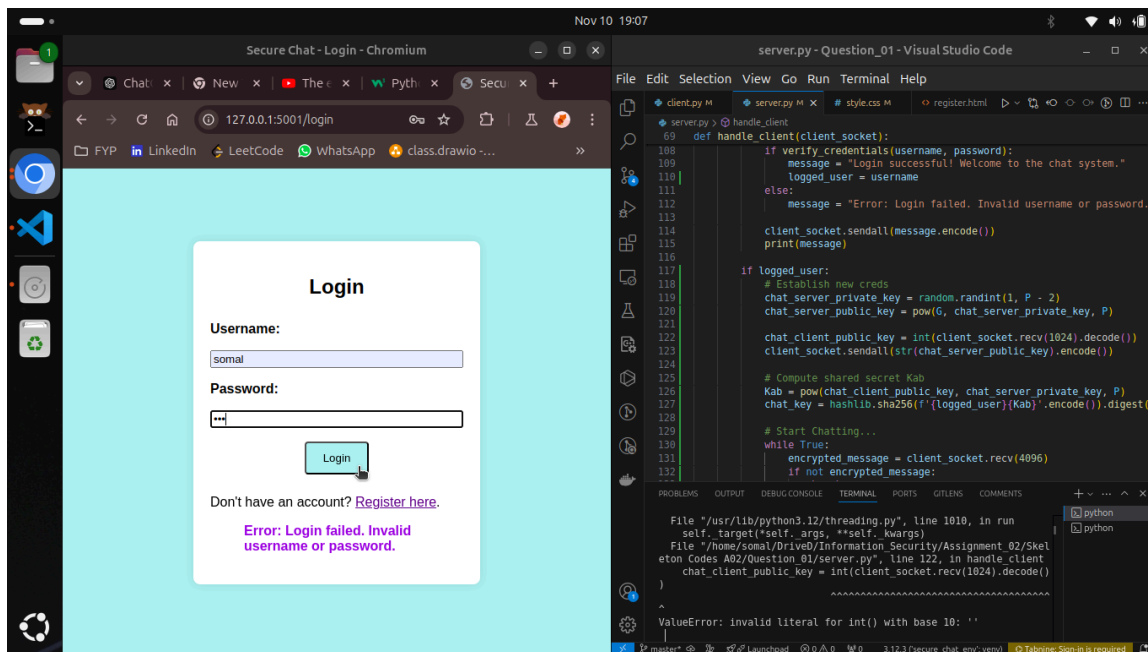
Enter valid credentials



Click on login and you will be redirected to the application after verification.



If user does not exist then it will show an error message



Function to verify passwords during login:

```
def verify_credentials(username, password):  
    with open('creds.csv', mode='r') as file:  
        csv_reader = csv.reader(file)
```

```

    for row in csv_reader:
        stored_username, stored_hash, stored_salt = row[1],
row[2], bytes.fromhex(row[3])
        if stored_username == username:
            hashed_input_password =
hashlib.sha256(password.encode() + stored_salt).hexdigest()
            return hashed_input_password == stored_hash
    return False

```

Encryption Testing

Capture the communication between the client and server using network analysis tools (e.g., Wireshark) and verify that all messages are encrypted in both Registration and Login phases including all messages exchanged. (50 marks)

Registration

The data from port 48214 → 5001 is to be monitored as server is running on 5000 and client is running on 48214

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays a project structure for 'QUESTION_01' with files like 'static', 'style.css', 'templates', 'chat.html', 'home.html', 'login.html', 'register.html', '.gitignore', 'client.py', 'creds.csv', 'README.md', 'requirements.txt', and 'server.py'. The Terminal panel at the bottom shows the execution of a Python script. The output indicates that the server is listening on port 48214, a client connected from 127.0.0.1, and the registration was successful after verifying the password and salted password.

```

File "/home/somal/DriveD/Information_Security/Assignment_02/Skeleton Codes A02/Question_01/server.py", line 159, in <module>
    while True:

File "/usr/lib/python3.12/socket.py", line 295, in accept
    fd, addr = self._accept()
KeyboardInterrupt

(secure_chat env) [somal@somals] - [~/DriveD/Information_Security/Assignment_02/Skeleton Codes A02/Question_01] - [Sun Nov 10, 19:09]
o [L[$] <git:(master*)> python server.py
Server is listening for connections...
Connected by ('127.0.0.1', 48214)
Password: 12345678
Salted Password: b'12345678oQW3'
Registration successful!

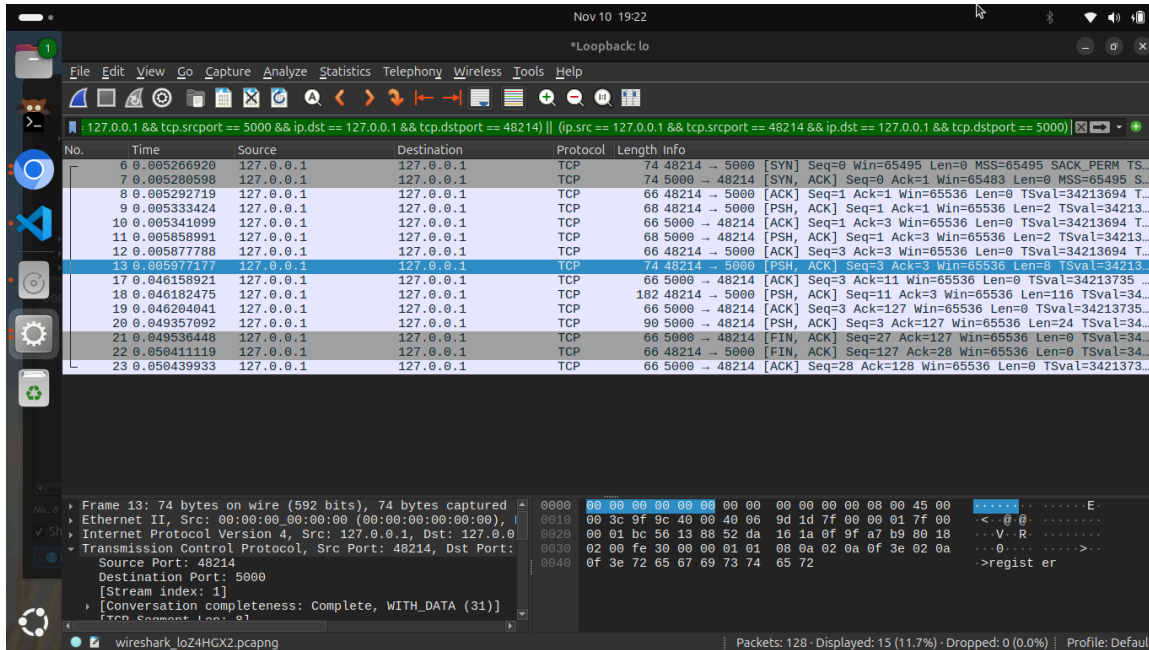
```

I applied following filter to capture relevant traffic

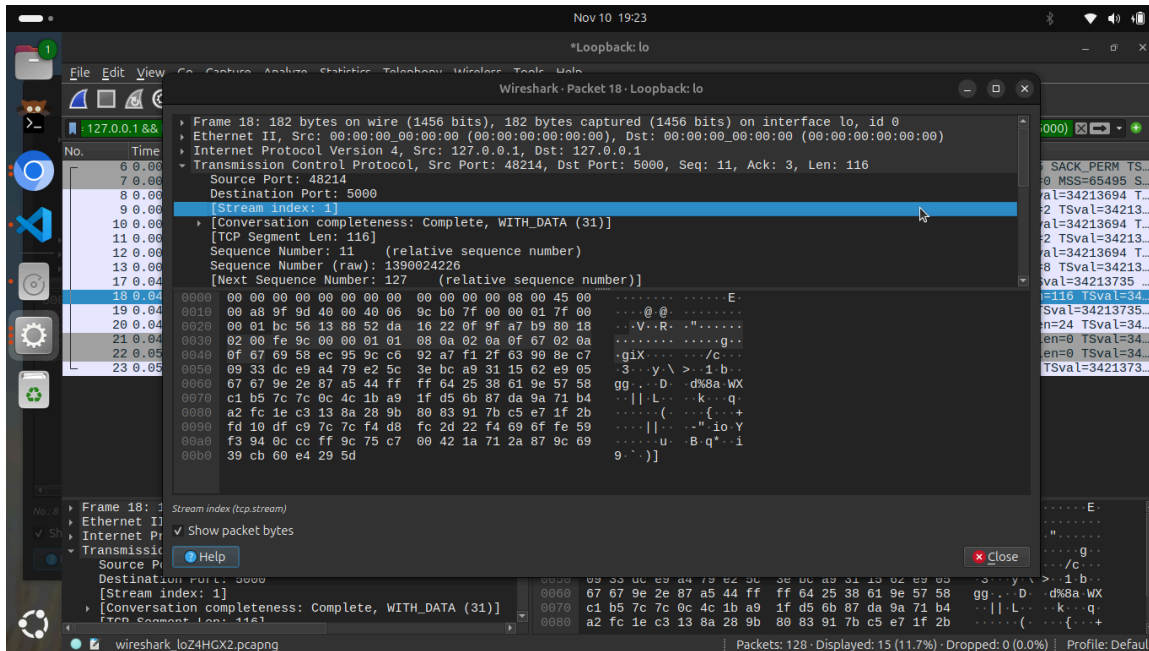
```

'(ip.src == 127.0.0.1 && tcp.srcport == 5000 && ip.dst == 127.0.0.1 && tcp.dstport == 48214) ||
(ip.src == 127.0.0.1 && tcp.srcport == 48214 && ip.dst == 127.0.0.1 && tcp.dstport == 5000)'

```



It is apparent that the data is well encrypted on observing all the packets one by one



Login

Now I am running wireshark again for login. This time the client port is 43480

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing files like `style.css`, `templates`, `chat.html`, `home.html`, `login.html`, `register.html`, `.gitignore`, `client.py`, `creds.csv`, `README.md`, `requirements.txt`, and `server.py`. The main editor displays a Python script with a `while True:` loop. The terminal at the bottom shows the execution of `python server.py`, which starts a server listening on `127.0.0.1`. It receives a connection from `127.0.0.1` on port `48214`, prompts for a password (`12345678`), and successfully registers and logs in the user.

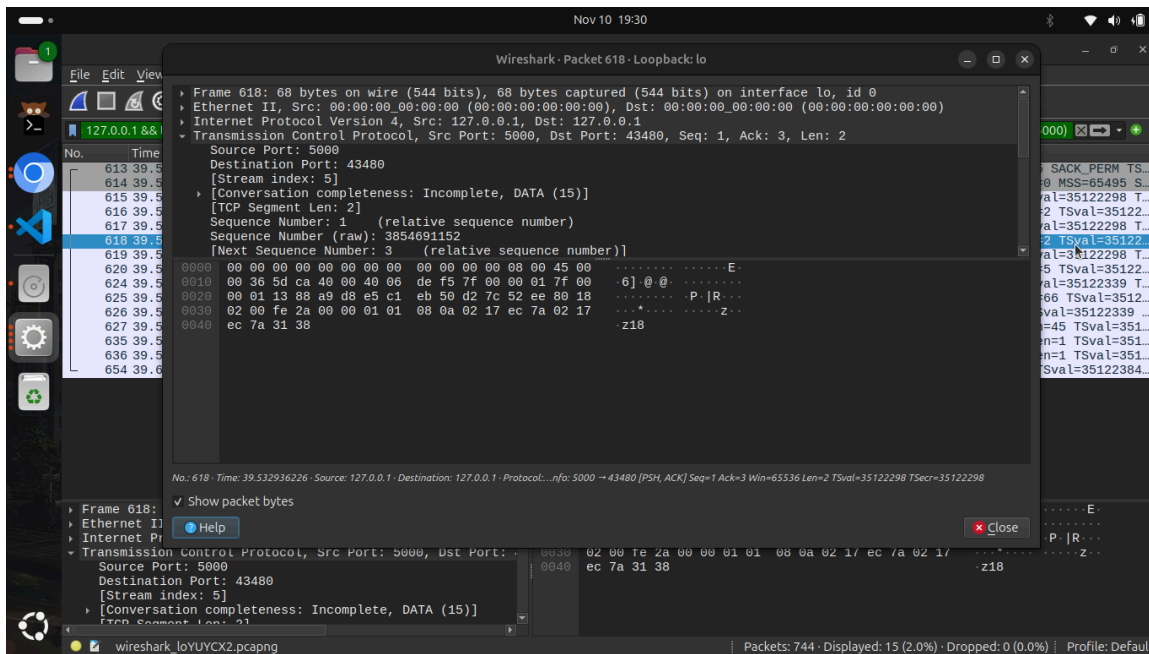
Upon applying filter we can observe the data:

`(ip.src == 127.0.0.1 && tcp.srport == 5000 && ip.dst == 127.0.0.1 && tcp.dstport == 43480) || (ip.src == 127.0.0.1 && tcp.srport == 43480 && ip.dst == 127.0.0.1 && tcp.dstport == 5000)`

The screenshot shows the Wireshark network protocol analyzer. The packet list pane displays a table of captured packets. The packet details pane shows the structure of a selected packet, including Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol fields. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
613	39.532545714	127.0.0.1	127.0.0.1	TCP	74	43480 → 5000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TS...
614	39.532562991	127.0.0.1	127.0.0.1	TCP	74	5000 → 43480 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 S...
615	39.532589130	127.0.0.1	127.0.0.1	TCP	66	43480 → 5000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3512298 T...
616	39.532637918	127.0.0.1	127.0.0.1	TCP	68	43480 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=2 TSval=35122...
617	39.532646359	127.0.0.1	127.0.0.1	TCP	66	5000 → 43480 [ACK] Seq=1 Ack=3 Win=65536 Len=0 TSval=3512298 T...
618	39.532936226	127.0.0.1	127.0.0.1	TCP	68	5000 → 43480 [PSH, ACK] Seq=1 Ack=3 Win=65536 Len=2 TSval=35122...
619	39.532951242	127.0.0.1	127.0.0.1	TCP	66	43480 → 5000 [ACK] Seq=3 Ack=3 Win=65536 Len=0 TSval=3512298 T...
620	39.532996979	127.0.0.1	127.0.0.1	TCP	71	43480 → 5000 [PSH, ACK] Seq=3 Ack=3 Win=65536 Len=5 TSval=35122...
624	39.573490402	127.0.0.1	127.0.0.1	TCP	66	5000 → 43480 [ACK] Seq=3 Ack=8 Win=65536 Len=0 TSval=35122339 T...
625	39.573508754	127.0.0.1	127.0.0.1	TCP	132	43480 → 5000 [PSH, ACK] Seq=8 Ack=3 Win=65536 Len=66 TSval=3512...
626	39.573521284	127.0.0.1	127.0.0.1	TCP	66	5000 → 43480 [ACK] Seq=3 Ack=74 Win=65536 Len=0 TSval=35122339 ...
627	39.573824076	127.0.0.1	127.0.0.1	TCP	111	5000 → 43480 [PSH, ACK] Seq=3 Ack=74 Win=65536 Len=45 TSval=351...
635	39.577495687	127.0.0.1	127.0.0.1	TCP	67	43480 → 5000 [PSH, ACK] Seq=74 Ack=48 Win=65536 Len=1 TSval=351...
636	39.577561397	127.0.0.1	127.0.0.1	TCP	67	5000 → 43480 [PSH, ACK] Seq=48 Ack=75 Win=65536 Len=1 TSval=351...
654	39.618511880	127.0.0.1	127.0.0.1	TCP	66	43480 → 5000 [ACK] Seq=75 Ack=49 Win=65536 Len=0 TSval=35122384...

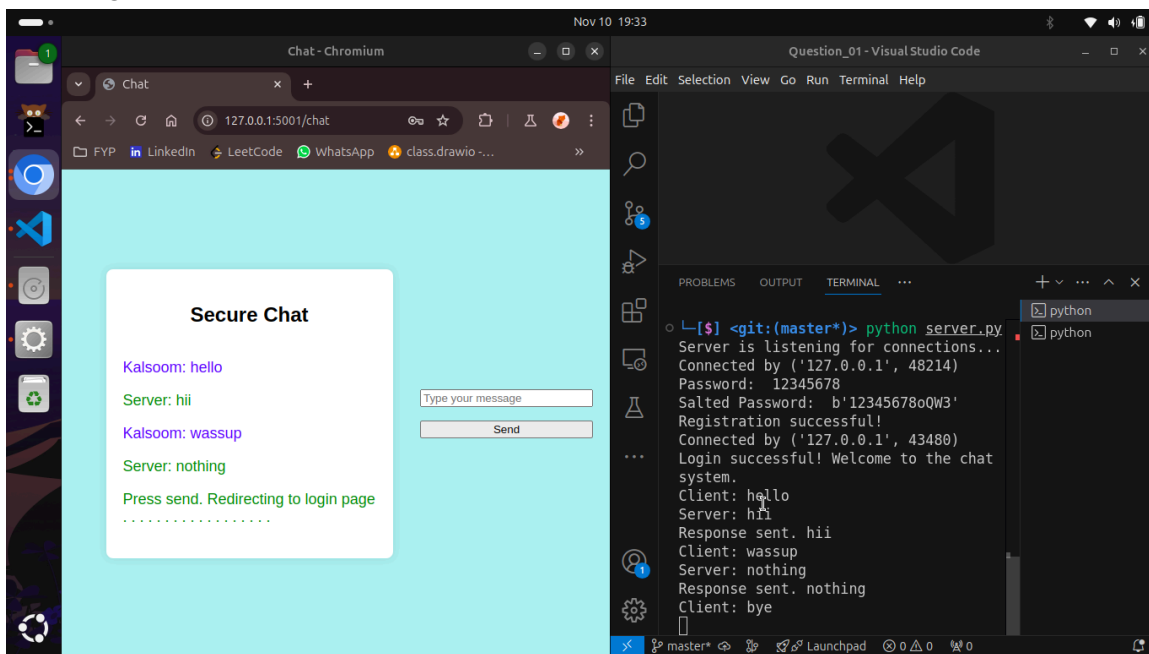
It is apparent that the data is well encrypted on observing all the packets one by one



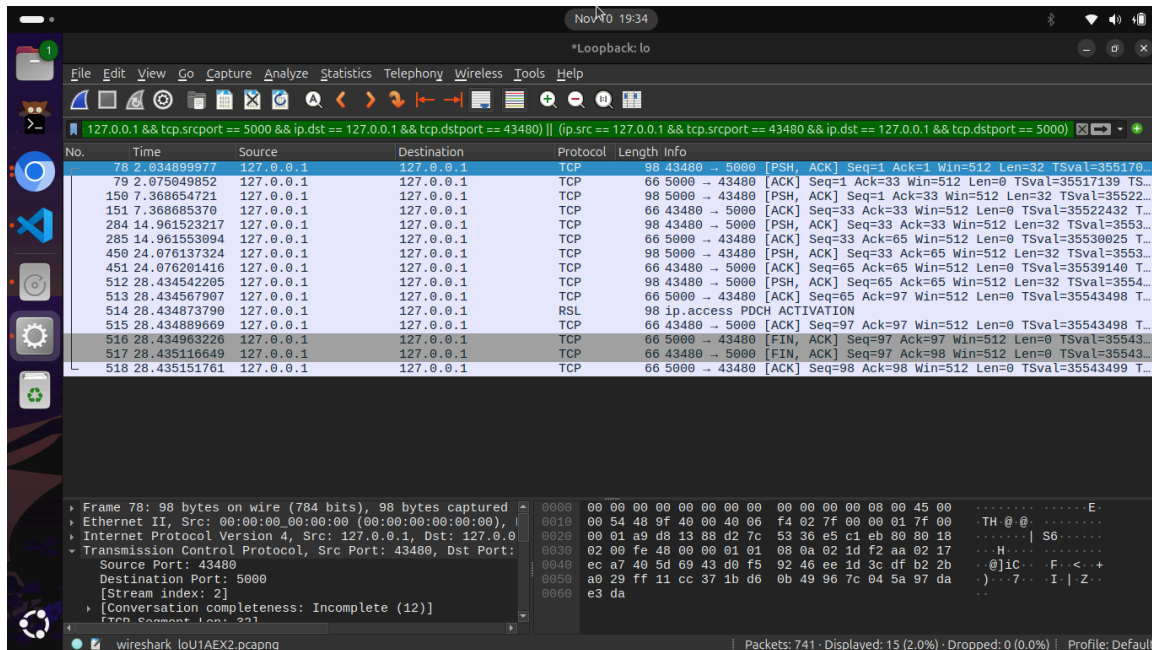
Chat

Similarly let's observe the chat conversation between client and server

Following is the conversation:



As the client and server communicate we are able to capture only the encrypted conversation.



It is apparent that the data is well encrypted on observing all the packets one by one

